GUESTGUARD

Inspector Quiz, API, and D1 Developer Guide

How question selection, attempts, grading, authentication, and persistence actually work.

Codebase snapshot: August 4, 2026

Short version: D1 does not grade anything by itself. The Cloudflare Worker grades in memory, then writes the attempt and result to D1. The certification answer key never ships with the public training site.

Audience: the next developer who has to change a question, debug an attempt, rotate an API URL, or figure out why a learner cannot retake the exam.

Contents

Heads up: This guide describes the repository as it exists now. The certification bank has 101 source questions; each initial exam serves 50.

1. The moving parts

There are three systems in play. Keeping them separate will save you a lot of confusion.

Piece	Where	What it owns
Training frontend	module5.html, module5.js, quiz-config.js	Renders questions, keeps an answer draft, sends authenticated API requests, and shows the result. It never receives canonical answer IDs.
Quiz Worker	quiz-worker/src/index.js	Authenticates the learner, selects questions, shuffles display IDs, grades submissions, enforces attempts, and writes D1.
D1	inspector-quiz-db	Stores quiz metadata, immutable attempt plans, submitted answers, scores, pass/fail state, and append-only result rows.
Question source	quiz-worker/src/question-bank.js	The 101 canonical certification questions and correct option IDs. This file is bundled into the Worker, not uploaded as a public site asset.

The certification quiz row in `quizzes` is mostly metadata: seed, title, and pass mark. Its JSON question and answer fields are empty placeholders because `quizBank()` switches that seed to the code-based question bank.

> **Heads up:** Editing the D1 questions JSON will not change the certification exam while the seed is inspector-certification-v1. Edit question-bank.js and redeploy the Worker.

Repository boundary

- The root Worker serves static training assets.
- The quiz-worker directory is excluded by .assetsignore, so the answer key and server code are not public training assets.
- R2 hosts video files only. It has no quiz or D1 binding.

2. End-to-end request flow

Starting an attempt

```
Browser Quiz Worker D1
POST /attempts/start -> validate Bearer token
{ quizSeed } hash issuer + subject
load quizzes row -> SELECT quiz metadata
find active attempt -> SELECT active
or load history -> SELECT submitted history
select + shuffle 50
INSERT OR IGNORE attempt -> quiz_attempts
<- public question text, shuffled display IDs, attemptId
```

Submitting an attempt

```
Browser Quiz Worker D1
POST /attempts/submit -> authenticate same learner
{ attemptId, answers } load owned attempt -> SELECT by id + learner_id
map display IDs to canonical IDs
compare with server answer key
claim active row atomically -> UPDATE ... status = active
append audit/result row -> INSERT results
<- score, total, rounded percent, pass/fail, missed question text
```

The display IDs are deliberately temporary. A browser sees IDs like `q7` and `q7o3`. D1 stores a question plan mapping those IDs back to canonical IDs such as `q032` and `o4`. That lets the Worker grade accurately without leaking the answer key in the start response.

3. D1 schema and row lifecycle

Table	Important columns	Purpose
quizzes	seed PK, title, questions, answer_key, pass_mark	Quiz configuration lookup. Certification questions come from Worker code despite the JSON columns.
quiz_attempts	id PK, learner_id, attempt_number, question_plan, status, score, passed, incorrect_ids, answers, timestamps	Source of truth for active and submitted attempts. One row per learner/quiz/attempt number.
results	id, seed, user_id, score, total, passed, answers, submitted_at	Simple append-only result record used for reporting or auditing.

Attempt state machine

```
new request -> active -> submitted
\-> never reopened

A passed history -> terminal complete response
Four submitted attempts without a pass -> terminal exhausted response
```

The unique index

`quiz_attempts_learner_number_idx` covers `(learner_id, quiz_seed, attempt_number)`. It is not optional. Two tabs can calculate the same next attempt number at the same time. `INSERT OR IGNORE` plus that index makes one row win; both callers then receive the canonical active attempt instead of creating duplicates.

Submission claim

The final update includes `WHERE status = 'active'`. The Worker checks `meta.changes === 1`. If two tabs submit together, only one claims the row; the other gets HTTP 409. This prevents double results and double grading.

4. Question bank and selection

Certification bank layout

Category	Bank size	Initial quota
GuestGuard purpose and inspector role	10	5
Workflow, remediation, and documentation	10	5
Exterior, access, pools, and pets	10	5
Fire safety and emergency evacuation	11	5
Electrical, utilities, detectors, and hazards	10	5
Accessibility measurements and judgment	10	5
Kitchen and appliances	10	5
Bathroom and hallway	10	5
Bedroom and common room	10	5
Overall property and guest experience	10	5

Initial attempt

`selectQuestions()` takes exactly five questions from each of ten categories, producing 50 questions. It independently shuffles each category selection and then shuffles the combined set.

Retakes

A retake size is twice the number missed on the previous attempt, capped at the bank size. Half of the slots target the same categories as the missed questions. The rest prefer questions the learner has not seen, then fall back to previously seen questions without duplicates.

> **Heads up:** A perfect or passing submission never creates a retake. A failed submission with zero missed questions is logically impossible, so selectQuestions() throws if asked to build one.

5. Grading and retakes

What actually grades

The Worker grades. D1 persists the outcome. The grading loop walks the stored question plan, finds the selected display option, maps it to the canonical option, and compares it with the canonical answer key from `question-bank.js`.

```
rawPercent = (score / questionCount) * 100
displayedPercent = Math.round(rawPercent)
passed = rawPercent >= Number(quiz.pass_mark)
```

The raw percentage decides pass/fail; rounding is display-only. This matters on short retakes. A raw 79.6 does not become a pass just because the UI displays 80.

Rule	Current value	Change location
Initial question count	50	INITIAL_QUESTION_COUNT in quiz-worker/src/index.js
Maximum attempts	4 total (initial + 3 retakes)	MAX_ATTEMPTS in quiz-worker/src/index.js
Pass mark	80 percent	quizzes.pass_mark in D1; seeded as 80 in schema/migration
Certification bank	101 canonical questions	quiz-worker/src/question-bank.js

What the response reveals

- Score, total, rounded percent, pass mark, remaining retakes, and whether another retake is allowed.
- Missed question number and text.
- No correct answer text and no canonical option IDs.

6. Authentication and API boundaries

Browser token

The portal redirects into training with a short-lived token. `training-api.js` exchanges it, stores the access token in browser localStorage, and `module5.js` sends it as `Authorization: Bearer ...`.

Worker validation

`authenticatedLearner()` forwards the bearer token to `AUTH_VALIDATION_URL`. Only a successful validation response is accepted. It then decodes the already-validated JWT, takes a stable subject claim, combines issuer and subject, and SHA-256 hashes them. D1 receives `portal:<hash>`, not an email address or raw token.

Case	HTTP result
No bearer token	401 Authentication required
Expired/invalid token	401 Invalid or expired authentication
Validation service unreachable	503 Authentication service unavailable
Attempt belongs to another learner	404 Unknown attempt
Second submission	409 already submitted
Origin not allowlisted	No Access-Control-Allow-Origin header

Heads up: The Worker currently validates against the URL in quiz-worker/wrangler.jsonc. Changing environments means changing AUTH_VALIDATION_URL and the allowed training origin, then redeploying. Do not casually point a test Worker at production identity services.

7. Function map

Function	Job
fetch(request, env)	Routes OPTIONS, /attempts/start, and /attempts/submit. Everything else is 404.
startAttempt(...)	Loads metadata/history, enforces terminal states, selects and inserts the next attempt, resumes an active attempt.
submitAttempt(...)	Validates ownership/payload, grades, atomically submits, inserts results, returns safe feedback.
selectQuestions(...)	Balanced first attempt or targeted retake selection. Exported for direct testing.
buildPlan(...)	Assigns temporary question/option display IDs and shuffles options.
publicQuestions(...)	Removes canonical IDs and answer data from the browser response.
attemptResponse(...)	Rebuilds the public payload when resuming an active attempt.
categoryCoverage(...)	Returns category counts for diagnostics/UI metadata.
quizBank(...)	Chooses the code bank, demo bank, or D1 JSON bank based on seed.
seededRandom(...) / shuffle(...)	Repeatable selection/shuffle helpers using the per-attempt variant seed.
authenticatedLearner(...)	Validates bearer auth and derives the hashed learner ID.
corsHeaders(...) / json(...)	CORS allowlist and hardened JSON response headers.

8. Safe edit recipes

Change or add a certification question

```
// quiz-worker/src/question-bank.js
question('q102', 'fire', 'Question text?', [
'Wrong answer',
'Correct answer',
'Wrong answer',
'Wrong answer'
], 1) // zero-based correct option index
```

- Use a permanent, unique canonical ID. Never renumber old IDs after attempts exist.
- Use one existing category ID from CURRICULUM_CATEGORIES.
- The final number is zero-based: 0 means first option, 3 means fourth.
- Run the bank invariant check and Worker dry-run, then deploy the quiz Worker. A static-site deploy alone does not update the bank.

Change a question safely

Text edits are fine. Changing an answer after active attempts exist is trickier: their plan stores canonical IDs, but grading uses the current answer key. An in-flight attempt could be graded against the new answer. For a major correction, use a new quiz seed or coordinate an attempt cleanup rather than silently flipping the key.

Change category quotas

Edit `testCount` values and keep their sum equal to `INITIAL_QUESTION_COUNT`. Each category needs at least its quota. The Worker throws if either invariant fails.

Change pass mark

Update the D1 `quizzes.pass_mark` row. The frontend config also says 80, but the server response is authoritative. Keep the frontend copy aligned so labels and assumptions stay honest.

Do not do this

- Do not put the answer key in module5.js or any root static asset.
- Do not delete or reuse canonical question IDs.
- Do not remove the unique attempt index.
- Do not trust learnerId, score, pass/fail, or question IDs supplied by the browser.